QMCPACK SUMMER SCHOOL 2025

Argonne NATIONAL LABORATORY

QMCPACK GPU SUPPORT

QMCPACK

YE LUO
Computational Scientist
Argonne National Laboratory

July 22nd 2025

INTRO

- QMCPACK has flagship high-performance GPU acceleration support.
  - All the major vendors are supported (NVIDIA, AMD, Intel)
  - Feature complete
- Easy to build for using GPUs
  - See the manual for configuration options (often just -DQMC_GPU_ARCHS=target)
  - Build scripts for common supercomputers are in <repo>/config
- Easy to run
  - Almost no change in QMCPACK input files
  - Only need to fine tuning walkers for optimal GPU performance
- We are happy to help support new machines

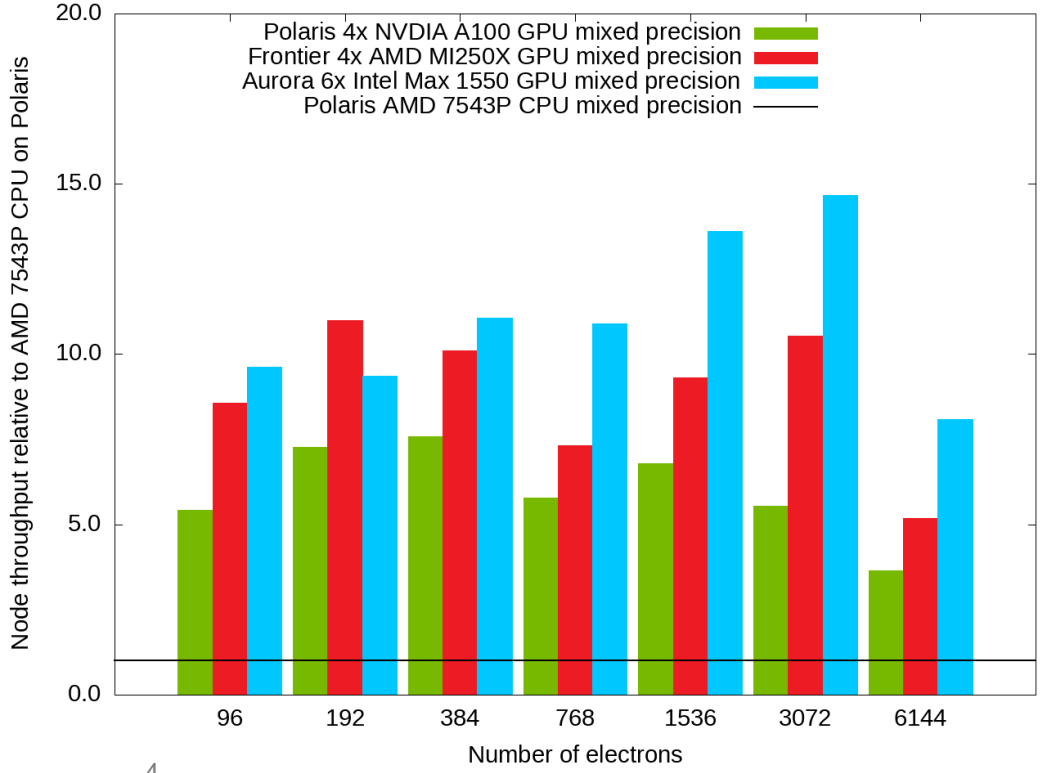
HISTORY

- 2010-2022 legacy CUDA code
  - Kenneth Esler introduced CUDA in 2010, last included in v3.16.
  - Enables a small set of QMC features with CUDA acceleration
- From 2017 performance portable code
  - Supported by Exascale Computing Project
  - Performant across NVIDIA, AMD, Intel GPUs
  - Uses portable programming model OpenMP
  - Feature complete design with CPU fallback
  - See *A High-Performance Design for Hierarchical Parallelism in the QMCPACK Monte Carlo code*. **DOI:** 10.1109/HiPar56574.2022.00008

PERFORMANCE ON EXASCALE MACHINES

works well on GPUs from all the vendors

Polaris node has 38.8 TFLOP in FP64 and 78 TFLOP in FP32. NVIDIA GPUs are the most efficient.
Aurora nodes are 312 TFLOP in FP64 and FP32, or 8x and 4x Polaris. Intel GPUs are still decent and competitive
Frontier
nodes are 191.6 TFLOP in both FP64 and FP32, or 4.9 and 2.5x Polaris. AMD GPUs can compete as well.



SUPPORTED GPUS AND SOFTWARE

All major vendors supported

- Hardware support includes NVIDIA, AMD and Intel GPUs
  - Data-center, workstation, desktop, laptop variants
- NVIDIA GPU needed software
  - LLVM >=17 for OpenMP offload support
  - Cudatoolkit for CUDA and accelerated libraries. >=12.3 recommended
  - NVHPC is not supported due to the insufficient OpenMP offload support.
- AMD GPU needed software
  - Upstream LLVM or downstream ROCm LLVM for OpenMP offload support
  - ROCm for HIP and accelerated libraries. >=6.3 recommended
- Intel GPU needed software
  - OneAPI HPC toolkit for both compilers and libraries. >=2024.0 recommended

GPU RELATED CMAKE OPTIONS

QMC_GPU and QMC_GPU_ARCHS

- Old user facing options removed since 4.0 release.
  - ENABLE_CUDA/ROCM/SYCL/OFFLOAD, QMC_CUDA2HIP
- QMC_GPU enables features using specific programming models
  - Semicolon separated list, candidates “openmp;cuda;hip;sycl”
  - “openmp;cuda” for NVIDIA, “openmp;hip” for AMD and “openmp;sycl” for Intel
- QMC_GPU_ARCHS selects GPU architectural targets
  - Semicolon separated list. Cannot mix targets from different vendors.
  - “sm_XX” for NVIDIA, “gfxXXX” for AMD and “intel_gpu_XXX” for Intel
- Recommendation to most users
  - Specify one entry to QMC_GPU_ARCHS based on your GPU
  - Leave QMC_GPU unset

CMAKE LINE EXAMPLES

- Common use cases
  - -DQMC_GPU_ARCHS=sm_80 for NERSC Perlmutter NVIDIA GPU A100
  - -DQMC_GPU_ARCHS=gfx90a for OLCF Frontier AMD GPU MI250X
  - -DQMC_GPU_ARCHS=intel_gpu_pvc for ALCF Aurora Intel GPU Max 1550

- Advanced use cases
  - -DQMC_GPU_ARCHS=sm_80 -DQMC_GPU=openmp. Overrides default “openmp;cuda” for testing without explicit CUDA source code.
  - -DQMC_GPU_ARCHS=“gfx906;gfx90a;gfx1030”. Creates a single executable working for multiple generations of AMD GPUs.

INPUT TAG

Avoid setting them unless being requested

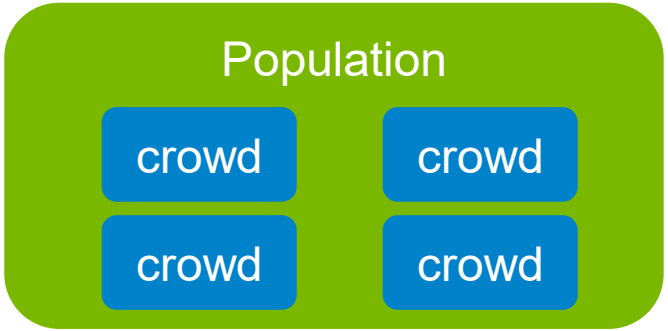
- ‘gpu’ tag. Accepted values: yes/no/cuda/sycl/omptarget/cpu
- Coarse selection gpu=yes/no. In a GPU build, pick the most performant implementation usually cuda/sycl, can be omptarget depending on each feature.
- Precise selection gpu=cuda/sycl/omptarget/cpu

```
<sposet_collection type="einspline" ... gpu="yes">
  <sposet name="spo-ud" size="8">
     <occupation mode="ground" spindataset="0"/>
  </sposet>
</sposet_collection>
```

CONCEPT OF CROWDS

Introduced in batched drivers

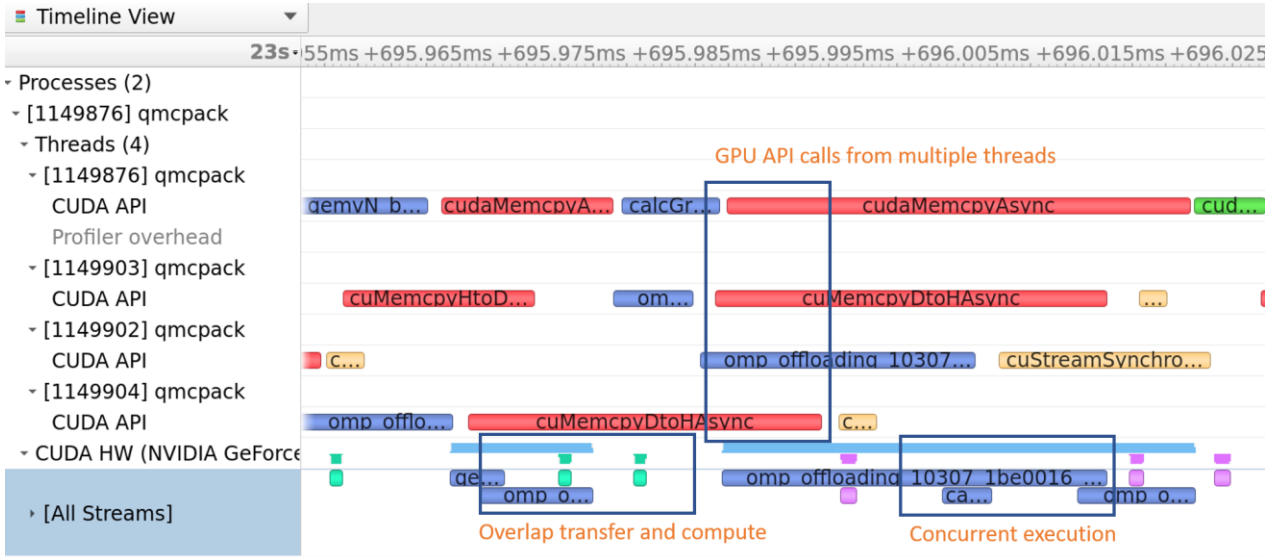
- The population within each MPI process(task) is divided into crowds. It is meant to maximize code performance.
- Crowds are mapped to CPU threads (4-8 preferred, default to #threads)
  - Having a few crowds enables efficient asynchronous computation.
  - Having too many crowds and too few walkers causes low efficiency
- [Debug] crowd_serialize_walkers=yes driver input to calculate one walker at time.



- lock-step walkers within a crowd
- Independent crowds (8 on Aurora)

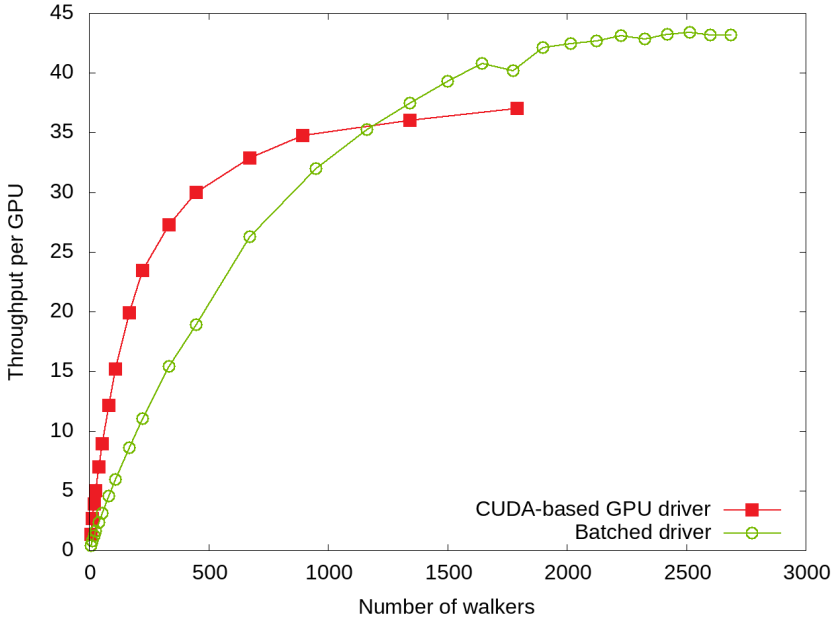
WHY ARE CROWDS NEEDED

- The assigned GPU is time shared by crowds and potentially concurrent execution.
- Overlap data transfer and computation.



WALKERS PER RANK

- Throughput definition
  - population x steps x blocks / driver time
  - Using walkers_per_rank here.
- More GPU resident walkers, more performance
- At high throughput domain, a few crowds help. In this case using 7.
- Make this study before massive production runs



CHOOSING THE NUMBER OF WALKERS

- walkers_per_rank vs total_walkers
  - “walkers” no longer accepted.
  - walkers_per_rank = MPI ranks x total_walkers must be satisfied.
  - Set one and the other is derived.
  - walkers_per_rank is set the number of crowds if neither provided. Recommended only in CPU only runs.
- Walkers on each rank is directly related to resources
  - CPU memory per MPI process
  - GPU on-board memory
  - Optimal value depends on the selected features

RUN SETTINGS

Node resource allocation

- Arguments of mpirun/mpiexec
- Number of MPI ranks per node
  - Simply the number of GPUs
- Bind each MPI rank to a set of cores
  - Divide the total number of cores by the number of GPUs.
  - Make sure assigned cores to each MPI rank stays in the same socket/NUMA
  - Avoid hyper-threads

- Perlmutter 64 cores with 4x A100 GPUs.
  - --ntasks-per-node=4
  - --cpus-per-task=16 or –c=16
  - --gpus-per-task=1
- Frontier 64 cores with MI250X containing 8x GCDs
  - --ntasks-per-node=8
  - --cpus-per-task=7 or –c=7
  - --gpus-per-task=1
  - --gpu-bind=closest

RUN SETTINGS

Node resource consumption

- Each MPI task gets a set of cores and one GPU.
- Control how QMCPACK uses them
- Environment variables controlling OpenMP behaviors
  - OMP_NUM_THREADS for the number of OpenMP threads <= CPUs per MPI task
  - OMP_PLACES, OMP_PROC_BIND controlling core affinity. Usually not needed.

- Perlmutter 64 cores with 4x A100 GPUs.
  - Using all the 16 cores isn't the best choice. Simply use 4.
  - export OMP_NUM_THREADS=4
- Frontier 64 cores with MI250X containing 8x GCDs
  - --c=7 implicitly set OMP_NUM_THREADS to 7 probably is good enough

CPU-GPU NODE ARCHITECTURES

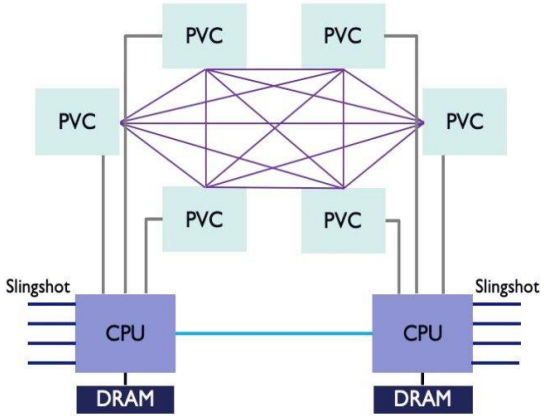
Aurora is complicated

0. One rank per NUMA
1. Put 1 MPI rank per PVC tile(GCD on AMD). Use 12 MPI ranks.
2. 52 CPU cores. Use 48 cores divisible by 6 per socket.
3. Don't bothered by indivisible network links

Argonne NATIONAL LABORATORY

Aurora Compute Node

- 2 Intel Xeon (Sapphire Rapids) processors
- 6 X^e Architecture based GPUs (Ponte Vecchio)
  - All to all connection
  - Low latency and high bandwidth
- 8 Slingshot Fabric endpoints
- Unified Memory Architecture across CPUs and GPUs



Overview of the Argonne Aurora Exascale System
2:30pm - 3:30pm, Feb 5
Legends Ballroom

U.S. DEPARTMENT OF ENERGY Argonne National Laboratory is a U.S. Department of Energy laboratory managed by UChicago Argonne, LLC.

AURORA AFFINITY SETTINGS

```
NNODES=`wc -l < $PBS_NODEFILE`
NRANKS=12     # Number of MPI ranks per node
NDEPTH=8      # Number of hardware threads per rank, spacing between MPI ranks on a node

NTOTRANKS=$(( NNODES * NRANKS ))
CPU_BIND=list:1-8:9-16:17-24:25-32:33-40:41-48:53-60:61-68:69-76:77-84:85-92:93-100
mpiexec -np ${NTOTRANKS} -ppn ${NRANKS} -d ${NDEPTH} --cpu-bind $CPU_BIND gpu_tile_compact.sh $exe_bin/qmcpack …
```

MEMORY USAGE REPORTS

- Look for the highest water mark
- The node has 1TiB CPU memory and each GPU stack has 64GiB dedicated VRAM
- “Available memory” is shared by all the ranks
- “Free memory on the default device” is exclusive to each rank.
- Rank 0 used 2703GiB and roughly 12 ranks use 32GiB

```
================================================
--- Memory usage report : VMCBatched after Warmup ---
================================================
Available memory on node 0, free + buffers : 1043132 MiB
Memory footprint by rank 0 on node 0      :    2703 MiB
Device memory allocated via OpenMP offload :     302 MiB
Device memory allocated via SYCL allocator :       0 MiB
Free memory on the default device         :   62844 MiB
================================================
```

MEMORY CONSIDERATION

In the case of B-spline orbitals

- The B-spline table is GPU resident and shared among all the walkers within the MPI process.
- Determinant related memory scales linearly to the number of walkers
- Matrix inversion is on GPU by default. matrix_inverter="host" to run on CPU and save the scratch space on GPU.
- <particleset gpu="no"> leave distance tables on CPU and thus saves GPU memory.
- Use hybrid basis to reduce spline memory usage.

SUMMARY

- QMCPACK GPU features are easy to enable in CMake
- CPU core affinity and GPU affinity are key settings for good GPU performance
- Making a walker scan for a specific system on a given machine is necessary to make a balanced choice between efficiency and time to solution.

U.S. DEPARTMENT OF ENERGY
Argonne National Laboratory is a U.S. Department of Energy laboratory managed by UChicago Argonne, LLC.

Argonne NATIONAL LABORATORY